# Reviewer Pack — Minimal Order of Categorial Invariants and Circuit Entanglem…

Entry ce7831557512 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 22:11:30 UTC

## Minimal Order of Categorial Invariants and Circuit Entanglement Bound

**Entry ID**: `ce7831557512` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 22:11:30 UTC

### 1. Conjecture

**Field A** (mathematical branch): Categorical Logic **Field B** (complexity object): Quantum Information Theory (Circuit Entanglement)

**Statement**:

> For every Boolean circuit C with n inputs, the minimal order of categorial invariants associated with its tautology set is linearly correlated with its entanglement entropy, such that Order(C) = Θ(Ent(C)).

**Rationale (proposer's reasoning)**:

> Categorical logic provides a framework to reason about logical systems in an abstract manner, potentially revealing hidden structures in circuits that are not apparent through traditional methods. The correlation between categorial invariants and circuit entanglement could expose a deep connection between logical structure and quantum information processing.

**Taxonomy category**: `CATEGORICAL_LOGIC_CIRCUIT_ENTANGLEMENT` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `2bfaec71c17ae1b1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if at least 80% of the generated circuits (n ≤ 40) have an absolute difference between the minimal order of categorial invariants and entanglement entropy less than or equal to 1, AND there are no seeds where this difference exceeds 10.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | $LLM_1$ | $LLM_2$ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "categorical logic" AND "entanglement entropy" - "quantum information theory" AND "circuit entanglement" AND "categorial invariants" - "Boolean circuit" AND "minimal order of invariants" AND "linear correlation"

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def evaluate_circuit(circuit):
        stack = []
        for gate, *inputs in circuit:
            if gate == 'AND':
                b = stack.pop()
                a = stack.pop()
                stack.append(a and b)
            elif gate == 'OR':
                b = stack.pop()
                a = stack.pop()
                stack.append(a or b)
            elif gate == 'NOT':
                a = stack.pop()
                stack.append(not a)
        return stack[0]

    def tautology_set(circuit):
        n = len(circuit)
        inputs = list(itertools.product([False, True], repeat=n))
        tautologies = []
        for input_values in inputs:
            if evaluate_circuit(list(zip(['NOT'] * n + ['AND'] * (n - 1), input_values))):
                tautologies.append(input_values)
        return tautologies

    def minimal_order(circuit):
```

```
        tautology = tautology_set(circuit)
        order = {}
        for i in range(len(tautology)):
            for j in range(i + 1, len(tautology)):
                if all(tautology[i][k] == tautology[j][k] for k in range(len(tautology))):
                    continue
                diff = sum(1 for k in range(len(tautology)) if tautology[i][k] != tautology[j][k])
                if diff not in order:
                    order[diff] = 0
                order[diff] += 1
        return max(order.values())

    def entanglement_entropy(circuit):
        n = len(circuit)
        inputs = list(itertools.product([False, True], repeat=n))
        probabilities = [evaluate_circuit(list(zip(['NOT'] * n + ['AND'] * (n - 1), input_values)))
        entropy = 0
        for p in probabilities:
            if p > 0 and p < 1:
                entropy -= p * math.log2(p) + (1 - p) * math.log2(1 - p)
        return entropy

    n_max = 40
    instances_tested = 0
    total_diff = 0
    max_diff = 0

    for n in [5, 10, 15, 20, 30, 40]:
        for _ in range(5):
            circuit = []
            for _ in range(n):
                gate = random.choice(['AND', 'OR'])
                inputs = tuple(random.choice([False, True]) for _ in range(n - 1))
                circuit.append((gate,) + inputs)
            order = minimal_order(circuit)
            ent = entanglement_entropy(circuit)
            diff = abs(order - ent)
            total_diff += diff
            max_diff = max(max_diff, diff)
            instances_tested += 1

    conjecture_holds = all(diff <= 1 for diff in [total_diff / instances_tested])
    counterexample = "mapping_undefined" if not conjecture_holds else ""

    return {
        "metric_name": "Absolute Difference",
        "metric_value": total_diff / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,
    results = []
```

```
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_diff = sum(r["metric_value"] for r in results) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_diff) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_diff} std={std_dev} support_fraction={support_fractio
    elif any(r["counterexample"] == "mapping_undefined" for r in results):
        print("RESULT: INCONCLUSIVE mapping_undefined")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f858ad5.py", line 105, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f858ad5.py", line 81, in run_trial
    order = minimal_order(circuit)
            ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f858ad5.py", line 47, in minimal_order
    tautology = tautology_set(circuit)
                ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f858ad5.py", line 42, in tautology_set
    if evaluate_circuit(list(zip(['NOT'] * n + ['AND'] * (n - 1), input_values))):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6f858ad5.py", line 33, in evaluate_circ
    a = stack.pop()
        ^^^^^^^^^^^
IndexError: pop from empty list
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls**: 11

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 14423 |
| 2 | propose | ollama_remote | glm4:latest | 0 | 0 | 12280 |
| 3 | propose | ollama_remote | glm4:latest | 0 | 0 | 12337 |
| 4 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 9150 |
| 5 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8481 |
| 6 | novelty | ollama_remote | glm4:latest | 0 | 0 | 8757 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13968 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 12747 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13186 |
| 10 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 13548 |
| 11 | judge | ollama_remote | glm4:latest | 0 | 0 | 46579 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 165456 ms total latency. Provider mix: {'ollama_remote': 11}

*(full prompt+response transcripts available in `research/audit/ce7831557512.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ce7831557512.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/ce7831557512.tar.gz` (if generated)